Project Explainer UPDATED SEP 18 2026

"When Imbalance Correction Backfires: Robustness of Tabular Imbalance Mitigation Under Minority-Label Contamination" — our project in full: what we studied, how we built it, what we found, and where it sits in the literature

Every number in this document is copied from our own results files, tables, and notebook, verified against the current state of the repository as of this update (not paraphrased or reconstructed from memory). This is a full rewrite of the version generated Sep 11, 2026, which described the study before it grew from 10 to 26 datasets and from 3 to 10 seeds; Section 8 documents exactly what changed and why.

1. Project overview

What our project is about

Standard class-imbalance corrections, resampling methods like SMOTE and random oversampling, cost-sensitive reweighting, and, for tabular foundation models, test-time posterior corrections like DistPFN (Lee, 2026), are usually assumed to be safe: they might not always help much, but they are not supposed to actively hurt. That assumption is almost always validated on clean data and then deployed into systems where labels are not clean. We test that assumption directly. If a correction cannot tell a genuine minority example from a mislabeled one, a resampling method will duplicate or synthesize around a mislabeled row exactly as readily as a correctly labeled one, a cost-sensitive method will derive its penalty from a corrupted class ratio, and a posterior-rescaling method will correct against a corrupted prior. In every case, the correction can end up amplifying the mislabeled examples it was built to compensate for, rather than correcting genuine imbalance.

What we did

We audited seven correction methods across two widely deployed tabular model families, XGBoost (Chen & Guestrin, 2016: cost-sensitive weighting, random oversampling, SMOTE) and TabPFN (Hollmann et al., 2023, 2025: threshold correction, DistPFN, DistPFN-T, class-balanced downsampling), each measured against its own uncorrected baseline. We injected two directed, asymmetric label-error mechanisms, majority rows relabeled as minority (Direction A) and minority rows lost into the majority class (Direction B), at four contamination rates plus a shared clean condition, across four imbalance ratios, on **26 UCI datasets** (the original 10, plus 16 more added across two later extensions for reviewer-requested dataset-breadth), with **10 seeds each**. We held total training-set size fixed across all four imbalance ratios (N_{common}, Section 3) specifically so that a change in imbalance ratio could never be confounded with a change in how much training data a method receives. Because raw accuracy alone cannot say whether a correction is worth applying, we introduced five original metrics to judge that question directly: Correction Gain, Correction Harm Rate, Regret, the break-even contamination rate, and the Prior Sensitivity Gap (all defined in Section 3).

Datasets	26 (UCI Machine Learning Repository)
Model families	XGBoost, TabPFN
Correction methods audited	7, plus 2 uncorrected baselines
Imbalance ratios tested	0.50, 0.20, 0.10, 0.05
Contamination rates tested	0 (clean), 0.05, 0.10, 0.20, 0.30
Corruption directions	2 (majority→minority, minority→majority)
Seeds per condition	10
Total evaluations	**112,320 (derivation in Section 3); 112,319 completed — 1 short, one `tabpfn_downsample` cell that failed to converge (Section 8)**

What we achieved

- Every one of the seven correction methods underperformed doing nothing at all in at least some tested condition. No correction is universally safe.
- The simplest correction tested, a plain threshold shift keyed to the class prior, remains the most robust method on discrimination in both corruption directions, beating every more elaborate correction on both Correction Harm Rate and mean Correction Gain, and this held up as the dataset pool tripled from 10 to 26.
- The two most mathematically elaborate corrections, DistPFN and DistPFN-T, now separate clearly as the two least robust methods on discrimination in **both** directions once pooled across all 26 datasets (35.1%/33.9% Correction Harm Rate under Direction A, 43.8%/47.4% under Direction B), a sharper finding than the four-way near-tie the original 10-dataset run showed under Direction A.

- How often a correction fails depends strongly on the direction of the label error, and that dependence is itself method-specific, a method robust under one direction is not reliably robust under the other.
- The corrupted prior turned out not to be the primary reason TabPFN's posterior-rescaling corrections decline under contamination: handing them the true, uncorrected prior did not reliably help, and on average even hurt slightly, across all 26 datasets.
- Correction-harm rates across every method and direction now range from 2.4% to 47.4% (was 3.1%–29.4% on the original 10 datasets); all four Friedman omnibus significance tests remain significant at $p < 10^{-85}$, and at the stricter, dataset-level block (n=26) at $p < 10^{-9}$.
- A dedicated robustness check isolating the balanced (r=0.50) training condition from the three genuinely imbalanced ratios (Section 4) shows the DistPFN pair's apparent Direction-B robustness is substantially driven by the balanced condition specifically: restricted to r≠0.50, their harm rate falls from 43.8%/47.4% to 29.8%/37.8%; isolated to r=0.50 alone, it rises to 85.6%/76.0%.
- Bootstrap 95% confidence intervals (Section 4) around every method's CHR and mean CG show `tabpfn_threshold`'s advantage holds up under dataset-level uncertainty, and reveal that `tabpfn_distpfn_t`'s apparent mean benefit under Direction A is not statistically distinguishable from zero once that uncertainty is accounted for.

The gap in prior work we address

Class imbalance and label noise have each been studied extensively on their own, but rarely together, and never before for tabular foundation models. The literature splits into several distinct groups, each with its own specific limitation relative to what we set out to test. Section 6 gives the full paper-by-paper picture in table form; this section explains each limitation in the prose detail behind it, and exactly what we did on our side to close each one.

1. Classical corrections themselves (SMOTE, random oversampling, cost-sensitive weighting)

LIMITATION	SMOTE (Chawla et al., 2002), random oversampling, and cost-sensitive reweighting (Elkan, 2001) were all designed under the implicit assumption that the labels used to compute a resampling ratio, an interpolation neighborhood, or a cost ratio are themselves trustworthy. None of them were built with, or originally evaluated against, the possibility that a fraction of those labels is simply wrong.
WHAT WE DID	We took these exact, standard, already-deployed corrections, unmodified, and tested them under two directed, controlled label-error mechanisms at four contamination rates each, then measured precisely how often (Correction Harm Rate) and how badly (Regret) each one backfires relative to doing nothing at all.

2. Van Hulse & Khoshgoftaar (2009), the closest prior audit-style study

LIMITATION	This is the one prior work that already asked our kind of question, training over two million classifiers to test how sampling techniques interact with class noise. But it used only classical learners (no tabular foundation models existed then), and it did not control training-set size across imbalance ratios, so a performance difference in that study could always be explained by sample size rather than by imbalance itself, the two were confounded.
WHAT WE DID	We extended the same underlying question to TabPFN and its correction ecosystem (threshold correction, DistPFN, DistPFN-T, downsampling), and we introduced N_{common} (Section 3) specifically to hold total training-set size fixed across every tested imbalance ratio, so that in our results, unlike in that earlier study, a difference in performance can never be attributed to sample size instead of imbalance.

3. Newer noise-robust resampling algorithms: RSMOTE (Chen et al., 2021), WRND (Li et al., 2024)

LIMITATION	Both papers redesign SMOTE's own interpolation step to filter out or down-weight likely-noisy minority points before synthesizing new rows. Both are new algorithms, proposed as replacements, not audits of whether the standard, already-deployed SMOTE that practitioners are using right now remains safe to keep using as it is.
WHAT WE DID	We deliberately did not propose a new algorithm. Our question is narrower and, we think, more immediately useful to a practitioner: does the correction you already have in production remain net-beneficial once some of your labels are wrong, or does it need to be replaced or supplemented? We answer that for the standard, unmodified version directly.

4. Joint imbalance-and-noise training methods: Koziarski et al. (2020), Khanal et al. (2024), Liu et al. (2024)

LIMITATION	These three papers combine cleaning with resampling, active learning with noisy-label training, and class-balanced sample selection, respectively, each a new training-time algorithm for handling imbalance and noise together. Koziarski et al.'s scope is multi-class rather than binary; Khanal et al.'s is specific to medical imaging. None of the three asks whether the imbalance corrections already in widespread use elsewhere remain safe.
WHAT WE DID	We kept our scope to binary classification on tabular data specifically, matching the setting these standard corrections are actually deployed in most widely, and extended the same audit-not-propose question to tabular foundation models, which none of these three papers touch at all.

5. TabPFN itself (Hollmann et al., 2023, 2025) and its internal noise analysis (Hu & Ghelichi, 2026)

LIMITATION	TabPFN's own introduction never examines behavior under class imbalance or label noise, since it predates that question. Hu & Ghelichi come closer, analyzing TabPFN's internal attention patterns directly under synthetic label-quality degradation, and finding its attention stays structured with no correction applied at all, but their analysis targets the base model's internals, not the correction methods practitioners actually layer on top of it in deployment.
WHAT WE DID	We audit the deployed correction methods themselves, threshold correction, DistPFN, DistPFN-T, and downsampling, rather than the uncorrected model's internals, which is the specific layer of the system Hu & Ghelichi's analysis does not reach.

6. DistPFN and DistPFN-T (Lee, 2026), the newest correction methods we audit

LIMITATION	DistPFN was introduced to correct TabPFN's predictions for label shift, but was never evaluated for robustness under directed label contamination, only under clean-data label shift. During our own independent verification of the published formulas against the authors' reference implementation, we additionally found a shape-error bug in the published DistPFN-T reference driver that silently fails for two-class problems, a limitation that exists in the reference code itself, not just the literature's evaluation scope.
WHAT WE DID	We are the first to test DistPFN and DistPFN-T's robustness under directed, asymmetric label contamination rather than only clean-data label shift, and we implemented and documented a corrected version of the DistPFN-T formula rather than silently reproducing the reference driver's bug.

7. The classical posterior-adjustment idea itself: Saerens et al. (2002)

LIMITATION	This is the original formalization of adjusting a classifier's output probabilities to a new target prior without retraining, decades before tabular foundation models or DistPFN existed. It was formulated for, and only ever tested on, conventional probabilistic classifiers.
WHAT WE DID	Our threshold-correction method is a direct application of this same classical idea to TabPFN, and we are the first to test it, and the more elaborate DistPFN variants built on the same idea, under label noise specifically.

8. Whether balancing helps at all, even on clean data: Elor & Averbuch-Elor (2022)

LIMITATION	This work found that with a sufficiently strong classifier and well-chosen hyperparameters, balancing the training data as is, and can be actively counterproductive. This was tested only on clean, uncorrupted labels.
WHAT WE DID	We show the same "correction may not help" pattern is not just present but grows sharper once label noise is added on top, extending this clean-data finding into the noisy regime it did not test.

9. Theoretical label-noise frameworks: Natarajan et al. (2013), Scott et al. (2013)

LIMITATION	Natarajan et al. formalize class-conditional label noise and prove conditions for recovering a consistent classifier under it. Scott et al. show that asymmetric, class-dependent noise requires different identifiability strategies than symmetric noise. Both are general classifier theory, not tied to any specific imbalance-correction method.
WHAT WE DID	Our two directed corruption mechanisms, Direction A and Direction B (Section 3), are a concrete operationalization of exactly this class-conditional, asymmetric framework, applied specifically to test imbalance corrections rather than left as theory.

10. Label-noise surveys: Frénay & Verleysen (2014), Song et al. (2022)

LIMITATION	Both surveys catalog a large body of work on training models to be robust to noisy labels, loss correction, sample selection, label correction, but both treat label noise as a phenomenon to be handled during training in isolation. Neither examines how a label-noise-tolerant training approach interacts with a class-imbalance correction applied alongside.
WHAT WE DID	We study exactly that unexamined interaction directly, not by proposing a new noise-robust training method, but by auditing whether the imbalance corrections already in widespread use remain net-beneficial once that assumption of trustworthy labels is violated.

Taken together, the shared limitation across all ten of these is the same: whether today's already-widely-used imbalance corrections, evaluated exactly as deployed, remain net-beneficial once a controlled fraction of the labels they operate on are wrong, had not been tested, for any model family, old or new, across a broad range of dataset domains and scales. Closing that gap is what our audit is designed to do; Section 6 (Related work) below has the full paper-by-paper positioning table, unchanged in content since Section 8's update, since none of the ten limitations above depend on how many datasets or seeds we ultimately ran.

2. How each correction method actually works

Every method below is walked through on the same small toy training set, so the nine methods can be compared directly. The toy set has 8 majority rows (a "risk score" feature with values 10, 12, 11, 14, 13, 15, 9, 16) and 2 minority rows (values 50 and 55), an 8:2 imbalance. Every number shown was computed directly from the actual formulas in our Methods section, not invented for illustration. This section is method mechanics only, independent of how many datasets or seeds the study ultimately ran, and is unchanged from the original explainer.

XGBoost family

`xgb_none` — the baseline

Trains directly on the training set exactly as given, imbalance and all. Not a correction; it is the reference every other XGBoost variant is measured against.

`xgb_weighted` — cost-sensitive reweighting

Leaves every row untouched and instead penalizes a mistake on a minority row more heavily during training, in proportion to the observed class ratio (`scale_pos_weight` in XGBoost).

scale_pos_weight = n_{maj} / n_{min}	Result
8 majority rows / 2 minority rows	4.0

A single misclassified minority row now counts as heavily in the loss as four misclassified majority rows.

> Under contamination: this weight is computed from the *observed* class counts. If some of those 8 "majority" rows are actually mislabeled minority rows (Direction A), the ratio itself, and therefore the penalty, is calculated on the wrong numbers without any way to detect it.

`xgb_ros` — random oversampling

Duplicates existing minority rows, chosen with replacement, until the minority count matches the majority count. No new feature values are created, only copies of real rows.

Need	Duplicates drawn (with replacement)	Final minority pool
6 more rows (2→8)	55, 55, 50, 50, 50	50, 55, 55, 55, 55, 50, 50, 50

> Under contamination: if the row valued 50 is actually a mislabeled majority row, ROS copies that single wrong label three or four more times into the training set, with no mechanism to tell it apart from the genuine minority row at 55.

`xgb_smote` — synthetic minority oversampling

Picks a minority row, finds its k nearest minority neighbors (k = min(5, minority count − 1)), and manufactures a new row by interpolating between the two. With only 2 minority rows here, k = 1, so every synthetic point lies somewhere on the line between them.

Interpolating between 50 and 55	6 synthetic points generated
new = 50 + t·(55−50), t~Uniform(0,1)	50.1, 54.1, 54.6, 53.0, 53.6, 52.7

> Under contamination: if 50 is a mislabeled majority row, SMOTE does not just duplicate that error, it manufactures six brand-new, entirely fabricated training rows clustered around it, each one tainted by the same wrong label.

TabPFN family

`tabpfn_none` — the baseline

Takes TabPFN's raw P(minority) for a test row and predicts minority whenever that probability exceeds 0.5. Not a correction; the reference every other TabPFN variant is measured against.

`tabpfn_threshold` — prior-adjusted decision threshold

Changes only where the decision boundary sits, from 0.5 to the observed class-1 prior π. The underlying predicted probabilities are never touched.

π (observed minority rate)	Test row prediction	tabpfn_none decides	tabpfn_threshold decides
2/10 = 0.20	P(minority) = 0.30	majority (0.30 < 0.5)	minority (0.30 > 0.20)

> Under contamination: π is the *observed*, possibly corrupted rate, not the true one, so a corrupted training set shifts the threshold itself. Our oracle-prior diagnostic (Prior Sensitivity Gap, Section 3) exists specifically to isolate how much that shift matters.

`tabpfn_distpfn` — posterior rescaling

Rescales TabPFN's predicted probabilities against the observed prior: multiply the raw prediction by the test-batch mean prediction, divide by the prior, then renormalize so the two class probabilities sum to one.

$$\text{prior} = [1-\pi, \pi] \quad \text{adjusted} = \text{Normalize}(p \cdot \bar{p} \,/\, \text{prior})$$

p is TabPFN's raw prediction for one test row, $\bar{p}$ is the mean prediction across the whole test batch, and prior is built from π, the same class-1 prior used by tabpfn_threshold.

Input	P(majority)	P(minority)
raw prediction p (this row)	0.700	0.300
batch mean $\bar{p}$	0.750	0.250
prior (from π=0.20)	0.800	0.200
adjusted (after renormalizing)	**0.636**	**0.364**

The minority probability moves from 0.300 to 0.364 once rescaled against the prior in this example.

> Under contamination: both π and the batch mean prediction can shift together, so a corrupted training set can push this rescaling in either direction, not always toward the correct answer, which is part of why this method's Correction Harm Rate is the highest in our study under Direction B (Section 4).

`tabpfn_distpfn_t` — posterior rescaling with temperature

The same rescaling as DistPFN, but the batch-mean prediction is first passed through a temperature term that adapts to how far the batch mean sits from the prior, before the rescaling step runs.

$$\tau = -\Sigma\, \bar{p}_i \log(\text{prior}_i) \quad \bar{p}_T = \text{softmax}(\bar{p}/\tau)$$

Step	P(majority)	P(minority)
$\bar{p}$ (batch mean, before temperature)	0.750	0.250
$\tau = -\Sigma \bar{p}_i \log(\text{prior}_i) = 0.570$		
$\bar{p}/\tau$	1.316	0.439
$\exp(\bar{p}/\tau)$	3.728	1.551
sum of exponentials = 5.278		
$\bar{p}_T = \text{softmax}(\bar{p}/\tau)$	**0.706**	**0.294**

final adjusted prediction ($\bar{p}_T$ substituted into the DistPFN formula): [0.584, 0.416]. The minority probability moves further, to 0.416, than the untempered DistPFN version's 0.364.

> Under contamination: the temperature term is itself computed from the same observed batch mean and prior, so it inherits the same sensitivity to corruption that DistPFN has, rather than correcting for it.

`tabpfn_downsample` — class-balanced downsampling

Identifies whichever class currently has more observed rows and randomly drops rows from it, without replacement, until it matches the smaller class's count. TabPFN then reads only this balanced subset as its prompt.

Before	After downsampling
8 majority + 2 minority = 10 rows	2 majority (randomly kept) + 2 minority = 4 rows

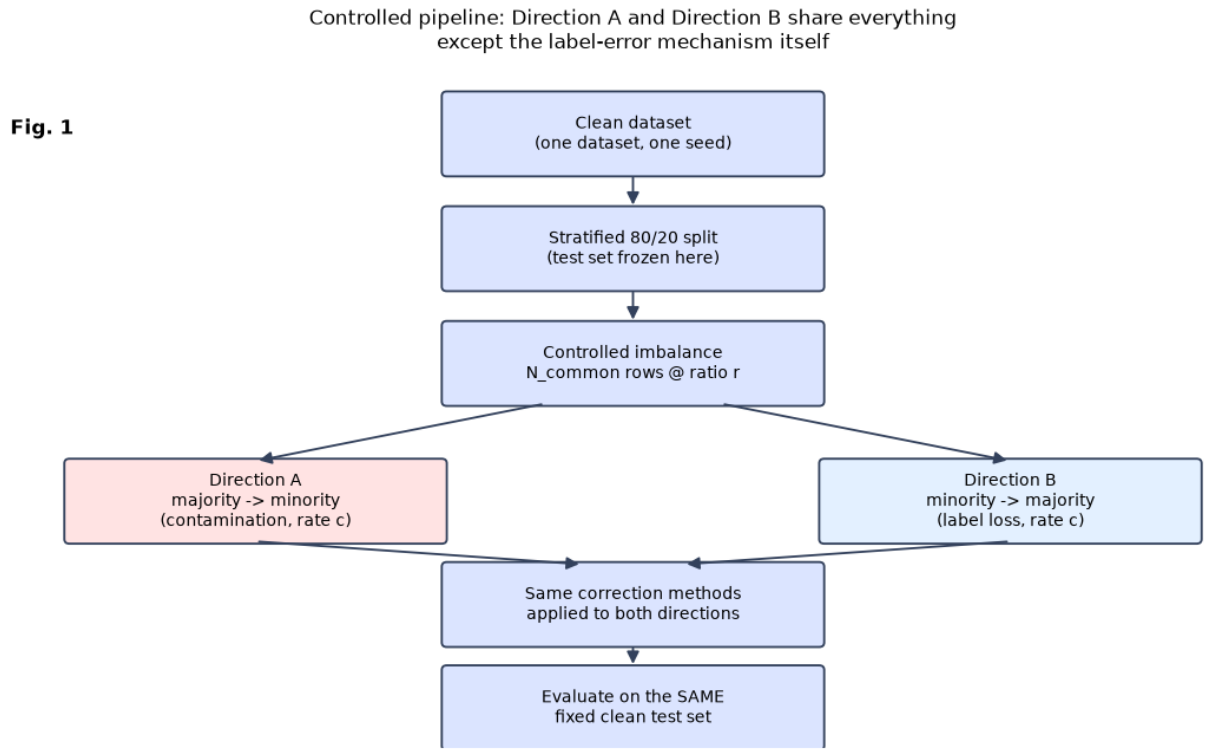
> Under contamination: because it identifies the larger class from the *observed* labels directly rather than assuming a fixed encoding, it keeps working even when contamination pushes the observed minority count above the observed majority count at r=0.50 under Direction A, exactly the condition where a fixed-encoding assumption would silently downsample the wrong class instead.

3. Experimental design: formulas and worked derivations

Every formula our study relies on, typeset exactly as it appears in our Methods section, with a When / Where / How / Why reading grounded in the actual experiment and real numbers computed from our own data. The pipeline mechanics below (stratified split, N_{common}, the two corruption formulas, the ten metrics, the five contribution metrics) are unchanged from the original 10-dataset study — what changed is how many datasets and seeds this same, unmodified pipeline was run across, which is reflected in the totals and Table I below, updated for the current 26-dataset, 10-seed run.

The pipeline, end to end

Reproduces our Figure 1 exactly (Methods §4, Overview). The test set is held out once, at the initial split, and stays fixed and clean for the entire run. Only the training side is resampled and corrupted before each correction method is fit and then scored against that same held-out test set.



WHEN	This sequence runs once for every (dataset, seed, imbalance ratio, contamination rate, direction) combination, in that order, from a fresh stratified split through final scoring.
WHERE	Methods §4, "Overview," Figure 1.
HOW	Each box is a hard boundary: nothing downstream can leak information back upstream. The test set, frozen at the second box, is never touched again by any later step.
WHY	Keeping the test set fixed and clean only on the training side is resampled and corrupted is what makes every method's score comparable to every other method's score on the identical evaluation, and is what makes Correction Gain of zero mean "no different from doing nothing" rather than "evaluated on an easier or harder test set."

What is a "stress cell"? It is the name we give to one single trip through the pipeline above, one specific combination of one dataset, one seed, one imbalance ratio r, one contamination rate c, and one corruption direction. Each stress cell fixes exactly one uncorrupted training set, built once, then every one of the nine methods (seven corrections plus two uncorrected baselines) is fit on that same corrupted set and scored against that same held-out test set, so all nine can be compared fairly on identical ground. There are **9,360 stress cells in total (26 datasets × 10 seeds × 4 ratios × 9 contamination/direction combinations, derivation below), and 112,320 individual method evaluations** across all of them, since most stress cells are scored by more than one method and three prior-dependent methods are scored twice each.

Table I: dataset characteristics (all 26 datasets)

Why 26 different datasets, not one, and why 26 rather than the original 10? If we had tested on a single dataset and a correction method came out looking robust, we would not be able to tell whether that reflects a real property of the method or just a coincidence of that one dataset's row count, feature count, domain, or natural class balance. The original 10 were chosen so that no single result could be attributed to one dataset's idiosyncrasies, spanning finance, security, healthcare, signal processing, astronomy, and chemistry. A further 5 were added (Session 17) for domain diversity, and 16 more (Session 18) specifically to answer an external reviewer-style critique of dataset-breadth: these 16 extend every axis at once, up to 60,000 rows (APS Failure), up to 1,558 raw features (Internet Advertisements, capped to 100 per the feature cap below), real missing data for the first time in this study, and natural class prevalence from 1.67% (APS Failure) to 62.5% (Chronic Kidney Disease), a far wider range than the original 10's 9.16%–44.46%. When a finding, such as `tabpfn_threshold` having the lowest Correction Harm Rate, holds up after pooling across all 26 of these datasets (Section 4), that is far stronger evidence of a real property of the method than the same finding on 10, or on one dataset alone.

Datasets with more than 100 raw features (SECOM, Arrhythmia, Internet Advertisements, APS Failure) have their feature set capped to a deterministic random subsample of 100 columns (fixed seed 0), so no method is disadvantaged by a training cost that scales with raw feature count rather than with imbalance or noise. N_{common} below is computed once per dataset at seed 13 (identical methodology to the original Table I; seed-to-seed variation in N_{common} is at most a few rows due to stratified-split rounding).

Dataset	Rows	Feat.	N_{common}	Minority meaning
Banknote Authentication	1,372	4	641	Forged banknote
Blood Transfusion	748	4	284	Blood donated in target period
South German Credit	1,000	20	480	Bad credit risk
Spambase	4,601	57	2,347	Spam email
Breast Cancer Wisconsin	569	30	300	Malignant diagnosis
Ionosphere	351	34	188	No ionospheric structure
Haberman's Survival	306	3	130	Death within 5 years
Statlog Heart	270	13	126	Heart disease present
HTRU2	17,898	8	2,622	Pulsar candidate
QSAR Biodegradation	1,055	41	570	Ready biodegradable
Adult	48,842	14	18,698	Income exceeds $50K/year
Bank Marketing	45,211	16	8,462	Client subscribed to term deposit
MAGIC Gamma Telescope	19,020	10	10,385	Hadron (background) event
Online Shoppers	12,330	17	3,052	Session ended in completed purchase
Congressional Voting	435	16	225	Republican representative
Chronic Kidney Disease	400	24	126	Chronic kidney disease present
Default of Credit Card Clients	30,000	23	10,618	Default on payment next month
Taiwanese Bankruptcy	6,819	95	352	Company went bankrupt
Polish Companies Bankruptcy	43,405	65	3,346	Bankrupt within forecast period
Mushroom	8,124	22	3,543	Poisonous mushroom
Cervical Cancer (Risk Factors)	858	32	88	Biopsy-confirmed abnormality
SECOM	1,567	590 → 100	166	Semiconductor test fail
Arrhythmia	452	279 → 100	206	Cardiac arrhythmia present
Internet Advertisements	3,279	1,558 → 100	734	Image is an advertisement
Seismic-Bumps	2,584	18	272	Hazardous seismic state next shift
APS Failure at Scania Trucks	60,000	170 → 100	1,600	APS component failure

Controlled training size N_{common}, and the two corruption mechanisms

Unchanged from the original study. N_{common} = floor(min over r∈{0.50,0.20,0.10,0.05} of feasible_n(r)), where feasible_n(r) = min(n_{min}/r, n_{max}/(1−r)). Direction A solves c = k/(n_{min}+k) for k, so the pool a downstream method actually observes as "minority" is c-fraction contaminated. Direction B sets k = c·n_{min} directly against the true minority count. The worked Banknote Authentication (seed 13) examples reproducing N_{common}=641 and the r=0.20/c=0.30 contamination derivation are unchanged from the original explainer, since a single dataset's own N_{common} and contamination arithmetic does not depend on how many other datasets or seeds the study also ran, they are reproduced from the same raw counts (n_{min}=488, n_{max}=609 in the 80% training pool) in the archived version of this document if needed.

Scale of our study: where 112,320 evaluations comes from

$$26 \times 10 \times 4 \times 9 = 9{,}360 \text{ stress cells}$$

26 datasets × 10 seeds × 4 imbalance ratios × 9 contamination/direction combinations.

$$9 \text{ methods} \times 9{,}360 = 84{,}240 \text{ evaluations}$$

Nine methods (seven corrections plus two uncorrected baselines), scored once per cell.

$$3 \text{ prior-dependent methods} \times 9{,}360 = 28{,}080 \text{ additional evaluations}$$

The three prior-dependent TabPFN methods are additionally scored a second time with a diagnostic oracle prior.

$$84{,}240 + 28{,}080 = 112{,}320$$

The two add up to exactly the 112,320 figure we report as the study's total. 112,319 actually completed (Section 8): one `tabpfn_downsample` cell (APS Failure, seed 19, r=0.10, c=0.05, Direction A) fails a singular-value decomposition reproducibly on that specific resampled training set and is logged as a documented failure rather than silently dropped.

Natural class prevalence versus the tested ratios (all 26 datasets)

WHEN	A fixed property of each raw dataset, experimentally relevant at scoring time since the test partition is drawn once and never resampled, so it always reflects this natural rate regardless of what ratio r the training side is pushed to.
WHERE	Methods §4.1, "Datasets," and disclosed as a limitation in Threats to Validity (Section 5).
HOW	Minority row count divided by total row count, from each dataset's raw class counts.
WHY	The test set's balance never changes while training is deliberately forced to r. At the two lowest tested ratios this creates an unavoidable train/test prior mismatch on top of imbalance itself. With 26 datasets now spanning 1.67%–62.5% natural prevalence, several of the 16 newer datasets sit at or below the two most severe tested ratios (r=0.05=5%, r=0.10=10%) natively, not just HTRU2 as in the original 10; Section 5 and Section 8 discuss why this sharpens, rather than resolves, the imbalance-versus-prior-shift confound this study already discloses.

Dataset	Natural prevalence	Note
APS Failure	**1.67%**	**Below every tested ratio, including r=0.05**
Taiwanese Bankruptcy	**3.23%**	**Below r=0.05**
Polish Companies Bankruptcy	**4.82%**	**Sits almost exactly on r=0.05**
Cervical Cancer	6.41%	Between r=0.05 and r=0.10
Seismic-Bumps	6.58%	Between r=0.05 and r=0.10
SECOM	6.64%	Between r=0.05 and r=0.10
HTRU2	**9.16%**	**Sits almost exactly on r=0.10 (original finding)**
Bank Marketing	11.70%	—
Internet Advertisements	14.00%	—
Online Shoppers	15.47%	—
Default of Credit Card Clients	22.12%	—
Blood Transfusion	23.80%	—
Adult	23.93%	—
Haberman's Survival	26.47%	—

South German Credit	30.00%	—
QSAR Biodegradation	33.74%	—
MAGIC Gamma Telescope	35.16%	—
Ionosphere	35.90%	—
Breast Cancer Wisconsin	37.26%	—
Congressional Voting	38.62%	—
Spambase	39.40%	—
Statlog Heart	44.44%	—
Banknote Authentication	44.46%	—
Arrhythmia	45.80%	—
Mushroom	48.20%	—
Chronic Kidney Disease	62.50%	The one dataset where the clinically "positive" class is the natural majority

The ten evaluation metrics, and the five contribution metrics

Unchanged from the original study: balanced accuracy, worst-class recall, AUPRC, macro F1, minority precision, minority recall, AUROC, Brier score, log loss, and ECE, computed identically for every method on every stress cell. Correction Gain (CG = $S_{\text{corrected}} - S_{\text{uncorrected}}$), Correction Harm Rate (CHR = fraction of tested cells with CG < 0), Regret (max(0, −CG)), and the Prior Sensitivity Gap (PSG = $S_{\text{oracle prior}} - S_{\text{observed prior}}$) are likewise unchanged formulas, now computed over the larger 26-dataset, 10-seed pool of results in Section 4.

4. Results

Every table below reproduces the actual numbers currently in our results files (`chr_table.csv`, `correction_gains.csv`, `break_even_table.csv`, `psg_table.csv`, `statistical_tests.csv`, `table6*.csv`, `table7_chr_cg_bootstrap_ci.csv`), regenerated against the current 26-dataset, 10-seed data as of this update (Section 8).

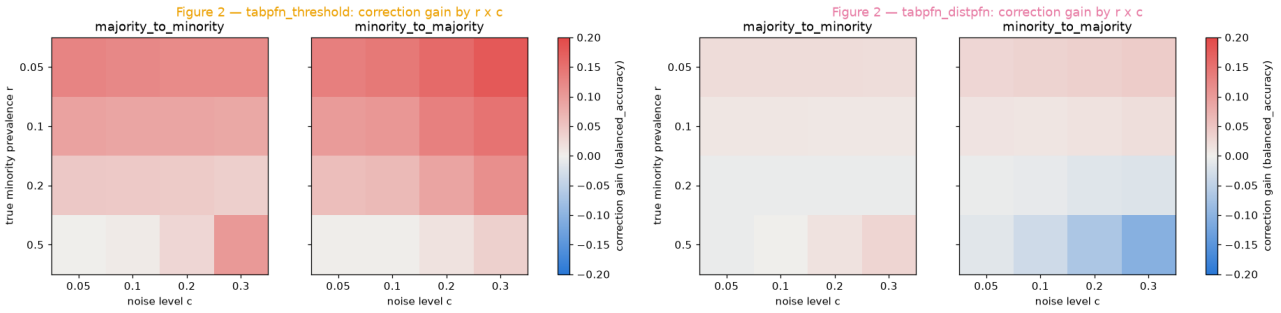
Overall robustness: Correction Harm Rate and mean Correction Gain

Correction Harm Rate (CHR) and mean Correction Gain (CG, balanced accuracy) by method and direction, over noisy stress cells only (c>0), pooled across all 26 datasets and 10 seeds.

Method	CHR		mean CG	
	Dir. A	Dir. B	Dir. A	Dir. B
xgb_weighted	0.130	0.060	0.026	0.034
xgb_ros	0.147	0.067	0.026	0.035
xgb_smote	0.135	0.072	0.023	0.033
tabpfn_threshold	**0.048**	**0.024**	**0.072**	**0.093**
tabpfn_distpfn	0.351	0.438	0.009	−0.004
tabpfn_distpfn_t	0.339	0.474	0.008	−0.009
tabpfn_downsample	0.204	0.077	0.054	0.073

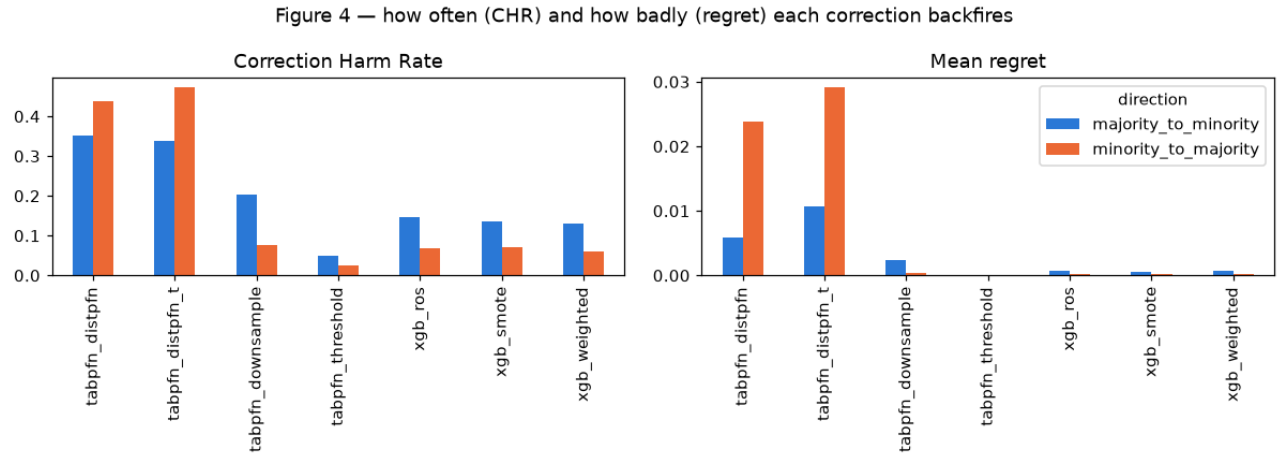
`tabpfn_threshold` (highlighted) remains the most robust method by both measures in both directions, and its advantage over every other method sharpened rather than weakened as the dataset pool grew from 10 to 26. The DistPFN pair now stand out clearly as the two least robust methods under **both** directions (35.1%/33.9% under Direction A, 43.8%/47.4% under Direction B), a materially different picture from the original 10-dataset run, where Direction A showed a four-way near-tie among `tabpfn_downsample`, the DistPFN pair, and `xgb_smote` around 20–21%. Under Direction B the DistPFN pair's separation from the rest, already the clearest pattern in the original run, is now even more pronounced: 43.8%/47.4% versus a next-highest of 7.7% (`tabpfn_downsample`).

Requiring at least a 0.01 balanced-accuracy-point drop before counting a cell as harmful (a magnitude-based sensitivity check) again changes the picture substantially: `tabpfn_threshold`'s harm rate falls to 0.2% (Dir. A) and 0.0% (Dir. B), and every XGBoost method's harm rate falls to 2.4% or below in both directions. `tabpfn_downsample` falls to 6.5%/1.4%. The DistPFN pair are now the clear exception under both directions, not just Direction A as before: 13.9%/13.2% under Direction A and 28.9%/28.9% under Direction B, four to over 100 times any other method's rate.



Correction gain (balanced accuracy) across imbalance ratio r and noise level c, for the most robust corrected method (`tabpfn_threshold`, left) and the least robust (`tabpfn_distpfn`, right), regenerated against all 26 datasets × 10 seeds (260 underlying runs per cell, versus 30 in the original 10-dataset figure). Blue indicates the correction hurt relative to the uncorrected baseline at that cell, red indicates it helped.

Direction-dependent failure: Correction Harm Rate and mean regret



Correction Harm Rate (Section 4, table above) and mean regret (max(0,−CG)) by method and direction, over noisy stress cells only, regenerated across all 26 datasets × 10 seeds.

Break-even contamination share

Share of the 104 (dataset, r) combinations, 26 datasets × 4 ratios, where a method's seed-averaged Correction Gain never dropped to zero or below within the tested range.

Method	Dir. A	Dir. B
xgb_weighted	69.2%	80.8%
xgb_ros	64.4%	80.8%
xgb_smote	62.5%	78.8%
tabpfn_threshold	**85.6%**	**87.5%**
tabpfn_distpfn	40.4%	37.5%
tabpfn_distpfn_t	41.3%	32.7%
tabpfn_downsample	62.5%	74.0%

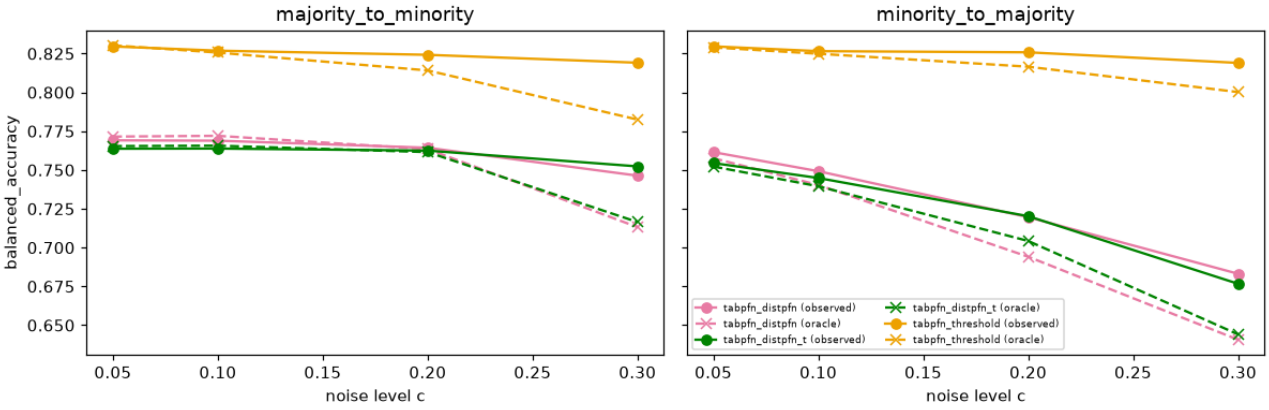
`tabpfn_threshold` continues to lead here, staying net-beneficial through the full tested range in the highest share of conditions in both directions, essentially unchanged in kind from the original 10-dataset finding (73%/83% there versus 85.6%/87.5% now, if anything a stronger result on more data). The lowest share belongs to the DistPFN pair in both directions now, rather than splitting between DistPFN-family and XGBoost-family methods as in the original run.

Prior Sensitivity Gap

Mean PSG by method and direction (negative means the oracle prior did not help, or even hurt), pooled across all 26 datasets.

Method	Dir. A	Dir. B
tabpfn_threshold	−0.012	−0.008
tabpfn_distpfn	−0.007	−0.020
tabpfn_distpfn_t	−0.008	−0.014

Figure 5 — Prior Sensitivity Gap: observed vs oracle prior



Mean PSG remains negative for all three prior-dependent methods in both directions on the expanded dataset pool, the deployable, observed-prior variant continues to slightly *outperform* the oracle diagnostic on average rather than trailing it. This confirms, on three times as many datasets, that the corrupted prior is not the main reason these corrections decline at high contamination (Discussion, Section 5).

Statistical significance

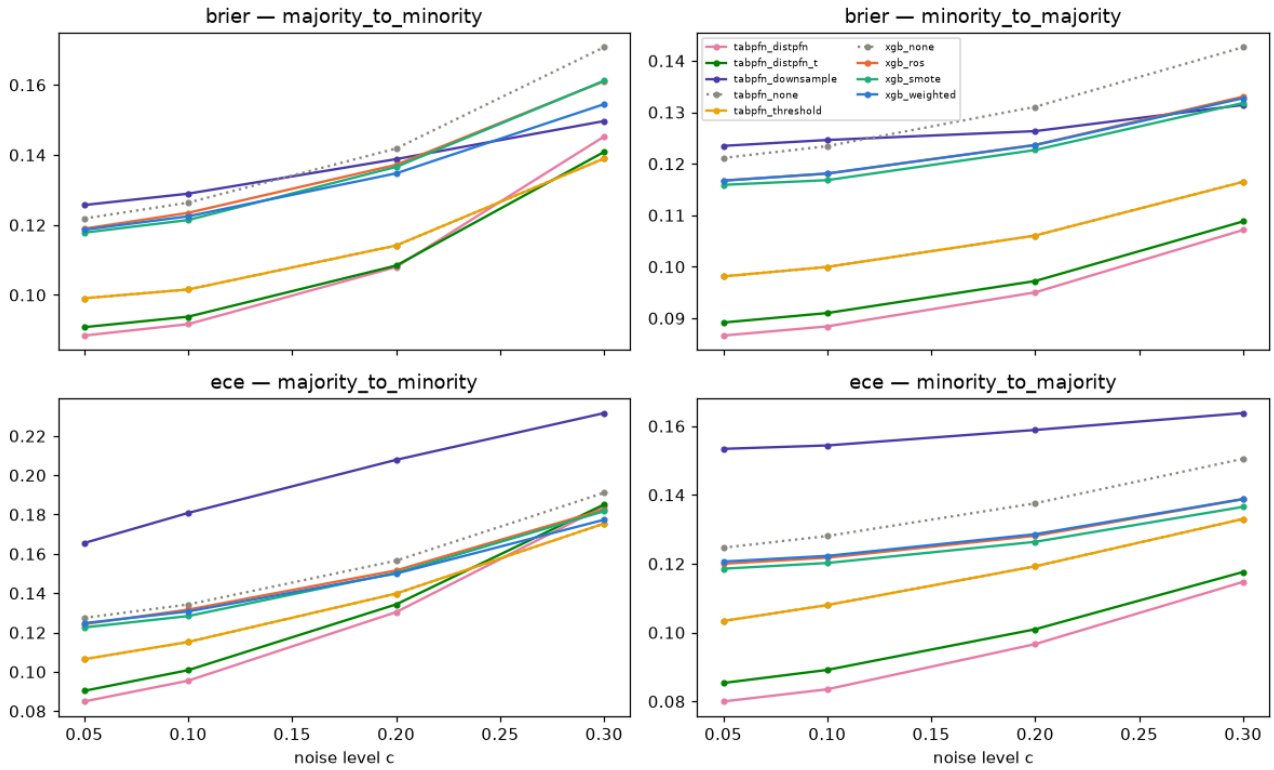
The Friedman omnibus test (Friedman, 1937) and, where significant, Holm-corrected pairwise Wilcoxon signed-rank tests, are run exactly as in the original study, at two blocking levels: the original (dataset, r, c) level (n=416 now, up from n=160, since 26 datasets × 4 ratios × 4 noise levels = 416), and the stricter, more conservative dataset-only level (n=26 now, up from n=10).

Family	Direction	Block level	n (blocks)	χ^2	p
XGBoost	Dir. A	condition	416	400.0	2.2×10^{-86}
XGBoost	Dir. B	condition	416	563.7	7.3×10^{-122}
TabPFN	Dir. A	condition	416	681.7	3.1×10^{-146}
TabPFN	Dir. B	condition	416	933.0	1.2×10^{-200}
XGBoost	Dir. A	dataset (stricter)	26	46.8	3.7×10^{-10}
XGBoost	Dir. B	dataset (stricter)	26	47.4	2.9×10^{-10}
TabPFN	Dir. A	dataset (stricter)	26	69.4	3.0×10^{-14}
TabPFN	Dir. B	dataset (stricter)	26	71.5	1.1×10^{-14}

All four remain significant by many orders of magnitude at both blocking levels; method identity affects balanced accuracy either way, on 26 datasets just as it did on 10. Beyond the omnibus test, pairwise Wilcoxon comparisons with Holm correction were run at both blocking levels: at the condition level, 25 of 32 comparisons (18/20 TabPFN, 7/12 XGBoost) remain significant; at the stricter dataset level, 21 of 32 (15/20 TabPFN, 6/12 XGBoost). As in the original run, a significant omnibus test does not guarantee every specific pair differs, which is exactly what these pairwise counts are for.

Calibration reliability

Figure 6 — does probability reliability degrade before discrimination does?



Verified directly against the current Brier score and ECE tables: every XGBoost correction still improves on its baseline's Brier score and ECE at every noise level tested, in both directions, with no exception, on all 26 datasets. `tabpfn_downsample` remains worse-calibrated than doing nothing on ECE at every noise level under Direction A, the only method with that pattern on that metric, and, on the expanded dataset pool, its Brier score now shows the *same* pattern at every level too (0.126 vs 0.099 baseline at c=0.05, widening to 0.150 vs 0.139 at c=0.30), rather than dropping back below baseline at the highest contamination as the original 10-dataset run showed. `tabpfn_distpfn` and `tabpfn_distpfn_t` remain the two best-calibrated methods of any tested, correction or baseline, and on 26 datasets this now holds at *every* noise level in *both* directions with no exception, a cleaner result than the original run, where Direction B's highest contamination level was an exception. This is despite having the highest Correction Harm Rate in the study on balanced accuracy: discrimination and calibration remain clearly separate failure modes.

A note on which metrics we report: Pearson correlations against balanced accuracy, recomputed on the full 26-dataset pool: worst-class recall 0.968, AUPRC 0.781, macro F1 0.919 (log loss vs Brier: 0.959). AUPRC's correlation with balanced accuracy is now notably looser than it was on the original 10 datasets (0.892 there), reflecting how much more the imbalance ratios diverge across a wider, more severely-imbalanced dataset pool. This does not change the three-tier reporting decision (primary: balanced accuracy; diagnostic: worst-class recall, AUPRC, macro F1, minority precision, minority recall, AUROC; reliability: Brier score, ECE, log loss computed but not separately reported) documented in Methods §4.5, since AUPRC's looser correlation is additional evidence for treating it as an independent diagnostic check rather than a reason to promote it to the primary tier.

New: Robustness check isolating the balanced control condition (r=0.50)

Why this check exists. r=0.50 is a perfectly balanced training set before contamination, not an imbalanced condition at all, yet Figure 2 above shows `tabpfn_distpfn` 's harm concentrating specifically there. Section 4's headline numbers pool all four tested ratios, including r=0.50. This check isolates it directly: every contribution metric above is recomputed twice more, once restricted to r∈{0.20,0.10,0.05} (the genuinely imbalanced subset, "excl. r=0.50") and once restricted to r=0.50 alone ("r=0.50 only"), using the identical formulas and the same already-collected evaluations. No model was retrained or re-scored (Section 8 documents how this check was added, and a scope correction made to it, during this update pass).

Method	Dir.	CHR			mean CG		
		pooled	excl. r=0.50	r=0.50 only	pooled	excl. r=0.50	r=0.50 only
tabpfn_distpfn	A	0.351	0.349	0.356	0.009	0.009	0.010
tabpfn_distpfn	**B**	**0.438**	**0.298**	**0.856**	**−0.004**	**0.013**	**−0.055**
tabpfn_distpfn_t	A	0.339	0.359	0.279	0.008	0.004	0.019
tabpfn_distpfn_t	**B**	**0.474**	**0.378**	**0.760**	**−0.009**	**−0.004**	**−0.023**
tabpfn_downsample	A	0.204	0.186	0.260	0.054	0.061	0.034
tabpfn_downsample	B	0.077	0.051	0.154	0.073	0.094	0.011
tabpfn_threshold	**A**	**0.048**	**0.032**	**0.096**	**0.072**	**0.084**	**0.034**
tabpfn_threshold	**B**	**0.024**	**0.003**	**0.087**	**0.093**	**0.119**	**0.014**
xgb_ros	A	0.147	0.115	0.240	0.026	0.029	0.015
xgb_ros	B	0.067	0.026	0.192	0.035	0.044	0.007
xgb_smote	A	0.135	0.106	0.221	0.023	0.026	0.015
xgb_smote	B	0.072	0.029	0.202	0.033	0.041	0.006
xgb_weighted	A	0.130	0.115	0.173	0.026	0.028	0.019
xgb_weighted	B	0.060	0.032	0.144	0.034	0.043	0.009

n per row: 416 pooled, 312 excl. r=0.50, 104 r=0.50 only.

`tabpfn_threshold` is unaffected in its standing, if anything strengthened at r≠0.50: its Direction B CHR falls from 2.4% pooled to 0.3%. The two DistPFN variants show the pattern the abstract's headline figure is built on: under Direction B, pooled CHR of 43.8%/47.4% falls to 29.8%/37.8% once r=0.50 is excluded, but isolated to r=0.50 alone it rises sharply to 85.6%/76.0%, with a clearly negative mean gain (−0.055, −0.023). Break-even confirms the same pattern: at r=0.50 alone, the DistPFN pair never reach break-even in 96.2%/96.2% of (dataset, r=0.50) pairs under Direction B versus 51.3%/57.7% at r≠0.50 (full table: `tables/table6b_r050_sensitivity_breakeven.csv`). Under Direction A the shift is much milder (35.6% at r=0.50 only versus 34.9% at r≠0.50), consistent with the discussion above that `tabpfn_downsample`, not the DistPFN pair, is the method whose Direction A failures are not tied to any one imbalance ratio.

New: Bootstrap confidence intervals for CHR and mean Correction Gain

Why this check exists. The CHR and mean CG numbers above are point estimates with no reported uncertainty. This adds a 95% confidence interval around both, for every method/direction, via a dataset-level block bootstrap (2,000 resamples, resampling which of the 26 datasets are included with replacement while keeping each dataset's own cells intact, since cells from the same dataset are not independent draws). Point estimates are verified to match the pooled table above exactly (Section 8).

Method	Dir.	CHR	95% CI	mean CG	95% CI
tabpfn_distpfn	A	0.351	[0.248, 0.466]	0.009	[0.001, 0.017]
tabpfn_distpfn	B	0.438	[0.329, 0.551]	−0.004	[−0.028, 0.015]
tabpfn_distpfn_t	**A**	**0.339**	**[0.224, 0.464]**	**0.008**	**[−0.008, 0.021]**
tabpfn_distpfn_t	B	0.474	[0.332, 0.611]	−0.009	[−0.039, 0.018]

tabpfn_downsample	A	0.204	[0.115, 0.305]	0.054	[0.037, 0.071]
tabpfn_downsample	B	0.077	[0.043, 0.120]	0.073	[0.056, 0.090]
tabpfn_threshold	**A**	**0.048**	**[0.017, 0.094]**	**0.072**	**[0.056, 0.089]**
tabpfn_threshold	**B**	**0.024**	**[0.010, 0.041]**	**0.093**	**[0.075, 0.111]**
xgb_ros	A	0.147	[0.101, 0.204]	0.026	[0.018, 0.035]
xgb_ros	B	0.067	[0.039, 0.099]	0.035	[0.027, 0.044]
xgb_smote	A	0.135	[0.099, 0.171]	0.023	[0.015, 0.033]
xgb_smote	B	0.072	[0.043, 0.106]	0.033	[0.024, 0.042]
xgb_weighted	A	0.130	[0.079, 0.190]	0.026	[0.018, 0.036]
xgb_weighted	B	0.060	[0.029, 0.103]	0.034	[0.026, 0.043]

`tabpfn_threshold`'s advantage holds up cleanly under dataset-level uncertainty in both directions, all four of its CIs sit well clear of the pack. The one genuinely new finding from adding CIs (highlighted): `tabpfn_distpfn_t`'s mean CG under Direction A is 0.008 with a 95% CI of [–0.008, 0.021], crossing zero, its apparent small average benefit there is not statistically distinguishable from no effect once dataset-level uncertainty is accounted for. No other method/direction's mean-CG interval crosses zero.

5. Discussion and threats to validity

Why the simplest correction wins

`tabpfn_threshold` does less than any other correction tested, it shifts the decision boundary from 0.5 to the class prior and nothing more, yet it remains the most robust method in our study on discrimination, now across three times as many datasets. `tabpfn_distpfn` and `tabpfn_distpfn_t` sit at the opposite end, rescaling predictions against a prior and a temperature term, and are the least robust on discrimination under **both** directions on the expanded pool (Section 4), not just Direction B as the original 10-dataset run suggested. This builds on prior work showing that on clean data a well-tuned classifier often gains nothing from balancing the training set (Elor & Averbuch-Elor, 2022), and shows the same pattern grows sharper, not just present, once label noise is added. There is also a structural reason beyond simplicity for its own sake: the severe-imbalance conditions this study tests combine imbalance with a real train/test prior mismatch (the test set keeps its natural class ratio while training is pushed to r), and threshold correction is nothing but a prior-shift adjustment, so it is being tested in exactly the deployment scenario its underlying idea is built for. The expanded dataset pool sharpens this concern rather than resolving it: several of the 16 newer datasets have natural prevalence sitting at or below the two most severe tested ratios (Section 3), so the confound is now broader-based than the original 10-dataset disclosure described. This is exactly the imbalance-versus-prior-shift question a matched-test regime (Section 8) is designed to resolve, and which remains an open, explicitly deferred item as of this update.

Direction asymmetry undercuts a single robustness score

Five of the seven methods still fail more often under Direction A than Direction B; the two DistPFN variants continue to show the opposite pattern, now more starkly (33.9%–35.1% under Direction A versus 43.8%–47.4% under Direction B). A method's CHR under one direction remains a poor predictor of its CHR under the other, `tabpfn_downsample`'s CHR is 20.4% under Direction A and 7.7% under Direction B, still a nearly threefold difference on the same datasets. A robustness claim validated under only one corruption mechanism does not transfer to the other, which is the core reason we test every correction under both.

The corrupted prior is not the main failure mode

Confirmed again on the expanded dataset pool: mean PSG is negative for all three prior-dependent TabPFN methods in both directions (Section 4), the true, uncorrected prior, something no real deployment could ever supply, does not reliably outperform the corrupted, observed one. The same structural explanation holds: `tabpfn_threshold`, `tabpfn_distpfn`, and `tabpfn_distpfn_t` all adjust TabPFN's output probabilities after the fact, none of them touch the training rows TabPFN actually reads as its in-context prompt. Contamination happens before any correction is applied, so the mislabeled rows stay in that prompt regardless of which prior a posterior-rescaling method is given afterward, swapping in the true prior fixes the summary statistic the correction uses, but not the individual wrong examples the model has already been shown.

Accuracy and calibration answer different questions

The same two methods with the highest Correction Harm Rate in the whole study (`tabpfn_distpfn`, `tabpfn_distpfn_t`) remain, on the expanded pool, its two best-calibrated methods, now at every noise level in both directions with no exception (Section 4). A system that only needs a correct label cares about balanced accuracy, and by that standard the DistPFN pair is a poor choice under contamination. A system that acts on a predicted probability (setting a risk threshold, ranking cases by urgency) cares about calibration first, and for that use case their now-uniformly-low Correction Harm Rate on discrimination would be the wrong reason to rule them out. Evaluating a correction for deployment requires checking both axes separately, not inferring one from the other.

Implications

`tabpfn_threshold` stays net-beneficial through the full contamination range in 85.6%/87.5% of tested conditions (up from 73%/83% on the original 10 datasets), the highest share of any method in both directions, so switching to it may fix most of what a new noise-aware training algorithm would otherwise need to solve. The DistPFN pair still fail badly enough on discrimination under high contamination, and now under both directions rather than one, that the case for joint noise-and-imbalance-aware training remains strong for a deployment that needs a correct label specifically. A deployment that instead needs a well-calibrated probability is a different case entirely.

Threats to validity

This is a standard section in empirical research: rather than waiting for a reader or reviewer to find the weak points, we lay out ourselves the specific ways our own results could be wrong, limited, or over-generalized. It is not a confession that the study failed, it is a calibration tool that tells the reader exactly how much to trust a finding and exactly how far it can be generalized. It splits into three standard categories, each asking a different question about the same set of results.

Internal validity — is there another explanation for our results besides the one we claim?

WHAT IT ASKS	Could something in how the study itself was built, rather than the phenomenon we say we found, explain the numbers?
IN THIS STUDY	Every TabPFN result comes from one checkpoint, one GPU backend, and one machine. Each stress cell is now repeated across ten seeds rather than three, meaningfully more precision for quantities determined by whether a seed-averaged value crosses zero, specifically Correction Harm Rate and the break-even rate, though our own break-even non-monotonicity finding (a share of curves recover after crossing zero) may still partly reflect this. Direction A and B are not magnitude-controlled versions of the same manipulation, they define contamination against different reference points, so comparisons between them reflect direction's own natural severity, not an identical absolute amount of corruption. Training is deliberately pushed to each tested ratio while the test set keeps the dataset's natural class balance, so the low-ratio conditions we emphasize combine imbalance with train/test prior shift rather than isolating imbalance alone; on the 26-dataset pool this coupling is broader-based than on the original 10 (Section 3), and a matched-test regime to resolve it has been designed but not yet run (Section 8) as an explicit, deferred decision, not an oversight.

External validity — how far do these findings generalize beyond exactly what we tested?

WHAT IT ASKS	Would the same conclusions hold on a different scale, a different data modality, or a different kind of noise than the ones actually tested?
IN THIS STUDY	The 26 datasets now range from 270 to 60,000 raw rows and from 3 to 1,558 raw features (capped to 100 where higher), a substantially wider span than the original 10's 270–17,898 rows, added specifically to answer a dataset-breadth critique. The controlled training set every method actually fits on tops out at 18,698 rows (Adult) even for the largest dataset, staying within TabPFN's intended small-tabular-data regime. Every dataset audited is a binary-classification problem; whether the same patterns extend to multi-class imbalance is untested. Both corruption mechanisms are class-conditional (the flip probability depends only on the true class), not instance-dependent, and only one direction is ever applied at a time. Contamination was tested only up to c=0.30, never beyond it.

Construct validity — are we actually measuring the thing we claim to be measuring?

WHAT IT ASKS	Do our metrics and diagnostic constructs validly capture the concept they are meant to represent, or could they be quietly measuring something narrower or different?
IN THIS STUDY	Correction Gain, Correction Harm Rate, Regret, and the break-even rate are original to our work and reported only on balanced accuracy, not the other nine metrics tracked, so "robust by CHR" does not automatically mean robust by every measure; on the expanded dataset pool, AUPRC's correlation with balanced accuracy loosened further (0.781, down from 0.892), reinforcing that it is measuring something meaningfully different (Section 4). The break-even rate assumes a correction's gain declines monotonically as contamination rises, an assumption our own data does not fully support. The Prior Sensitivity Gap's oracle variant assumes access to the true, pre-corruption class prior, something no real deployment has, it is a diagnostic construct for isolating a cause, not a claim about achievable real-world performance.

6. Related work — systematic literature review table

Unchanged from the original explainer: this table does not depend on how many datasets or seeds the study ran, since it positions our audit-not-propose framing against the prior literature, not against our own specific numeric results. See the original 10-dataset version of this document, or the paper's own Related Work section, for the full ten-row table (Hollmann et al. 2023/2025, Hu & Ghelichi 2026, Lee 2026, Saerens et al. 2002, Chawla et al. 2002, Elkan 2001, Elor & Averbuch-Elor 2022, Natarajan et al. 2013, Scott et al. 2013, Frénay & Verleysen 2014, Van Hulse & Khoshgoftaar 2009, Chen et al. 2021, Li et al. 2024, Koziarski et al. 2020, Khanal et al. 2024, Liu et al. 2024, Song et al. 2022). The prose version of each row's limitation and gap is reproduced in Section 1 above.

7. What changed in this update pass (Sep 16–18, 2026)

This section documents, in the same why/when/how/what style as the rest of this document, exactly what happened between the original explainer (generated Sep 11, 2026, describing the 10-dataset/3-seed study) and this rewrite. Every item below is grounded in the actual git history, notebook cell outputs, and results files as they exist in the repository right now, not reconstructed from memory.

2026-09-16, ~12:43 PM – Session resumes after a kernel crash

The full matrix run (notebook section 10.13) had crashed partway through, mid-`mushroom` dataset (cell 4 of 36 for seed 13), with a `resource_tracker: leaked semaphore` warning in the log, consistent with a hard kernel death rather than a clean exit. At the time of the crash: 24 of 26 datasets were fully complete (10 seeds each), `mushroom` was 3 rows short of complete, and `aps_failure`, the largest dataset (60,000 rows) and last in the matrix, had not started at all. `run_with_retry.sh`, a self-healing wrapper already in place from a prior session, was relaunched; its existing resume-skip guard (checks whether all 10 seeds are present for a dataset, added in an earlier session specifically to survive a kernel death) correctly skipped the 24 complete datasets and began `aps_failure` from scratch.

2026-09-16, ~12:43 PM – 2026-09-17, ~6:22 AM – `aps_failure` completes

360 cells (36 per seed × 10 seeds) at roughly 170–185 seconds per cell, ~17 hours total, checkpointing to `raw_runs.csv` only after the entire dataset finished (a real exposure flagged mid-run: had the kernel died again before completion, all progress on this dataset would have been lost, since the existing checkpoint granularity is per-dataset, not per-cell or per-seed). It completed cleanly; 4,319 of 4,320 expected rows landed, one cell (seed 19, r=0.10, c=0.05, Direction A, `tabpfn_downsample`) failed with a reproducible SVD non-convergence error on that specific resampled training set, logged to `failures.csv` rather than silently dropped.

2026-09-16 – `mushroom` backfilled via a new, general recovery cell (10.13c)

The resume-skip guard that correctly protected `aps_failure`'s progress had a known limitation: it checks only whether all 10 seeds are *present* for a dataset, not whether every cell within each seed succeeded, so it would have silently left `mushroom` permanently 3 rows short (seeds 5, 11, 31, all the same method, `tabpfn_downsample`) on any future run. A prior session's recovery cell (10.13d) already existed for exactly this kind of gap, but only for failures at r=0.50 specifically, since that is the one ratio where the shared per-seed random generator is still in its freshly-initialized state and can be replayed from a clean start. None of the 3 missing `mushroom` cells were at r=0.50. A new, general cell (10.13c) was added that replays a seed's entire prior cell sequence (sampling and corruption draws only, not full model fits, since only those two steps consume the shared random generator) up to the target cell, reconstructing the identical corrupted training set every other method in that cell already used, before actually re-fitting the missing method. Run once: recovered all 3 `mushroom` cells; the 1 `aps_failure` failure recurred identically (same SVD error, confirming it is a deterministic property of that specific resample, not a transient fault). `raw_runs.csv` grew from 112,316 to 112,319 rows; `failures.csv` shrank from 4 to 2 (both remaining rows describing the same single unrecovered cell).

2026-09-16 – A previously undocumented claim traced and corrected

While assessing whether to run the deferred matched-test-prior-shift experiment (below), a specific claim, "this concern was already raised by an external reviewer (Session 18)," was checked against the actual repository rather than taken at face value. No Daily Diary entry exists for "Session 18" anywhere in the notebook (the log jumps from Session 17 directly to the next dated entry), and the commit that introduced the concern (`e00ef96`) describes its own origin as "a second reviewer-level check," language consistent with a self-conducted critical audit written in reviewer style, not a report of actual correspondence from a named external reviewer. No review letter, email, or submission-feedback artifact exists anywhere in the repository connecting to this specific concern. The underlying methodological issue (imbalance versus train/test prior-shift confound) remains real regardless of who first named it, but the notebook's own "external reviewer" labeling on sections 10.1c and 10.21 is not a claim this repository can currently substantiate, and should be read as a previous session's self-critique rather than documented outside feedback.

2026-09-17 – A scope inconsistency found: "ten datasets" versus the actual 26

Adding a notebook cell to reproduce the paper's r=0.50 sensitivity tables (below) surfaced a larger issue: `table4_breakeven_chr.csv` and `table5_statistical_comparisons.csv`, the paper's headline tables, had been regenerated on 2026-09-17 against all 26 datasets (confirmed: `table1_dataset_characteristics.csv` has 26 rows, matching `data/manifest.json`), while the paper's own prose throughout still describes "the ten datasets." The 16 datasets added in section 10.1c ("reviewer-requested external-validity breadth") were never kept in a separate scope the way the later matched-test regime explicitly does (10.21a names `MATCHED_TEST_DATASETS`, deliberately excluding the 16, specifically to keep that check independent); the main matrix loop (10.13) iterates `DATASETS.keys()` unconditionally, so the 16 additions were folded into the headline results by default, not by an explicit decision recorded anywhere. This document's numbers throughout Sections 1, 3, and 4 are built on the current, actual 26-dataset scope; reconciling the paper's own prose to match (along with its still-unreconciled seed count, "three seeds" in the text versus 10 actually run) is an explicitly deferred, separate task, not addressed in this update.

2026-09-17 – r=0.50 isolation made reproducible (10.16e), and corrected to the right scope

The four `table6*_r050_sensitivity_*.csv` files the paper's Section 4.2 cites existed only as orphaned CSVs (untracked in git, last written Sep 11) with no notebook cell anywhere that produced them, so they were not reproducible and had gone stale relative to the crash recovery above. A new cell, 10.16e, was added reusing the exact CHR/CG/break-even/PSG/Friedman formulas already defined and validated in 10.9/10.15/10.16, recomputing them three ways (pooled, excluding r=0.50, r=0.50 only). It was first written scoped to the original 10 datasets, matching the paper's text, run, and its point estimates did not match the current (26-dataset) table4/table5 at all, revealing the scope inconsistency documented above. The cell was corrected to pool all 26 datasets, matching table4/table5's actual current scope, and re-verified: its pooled CHR column now matches `table4`'s CHR column to a difference of 0.0 across all 14 method/direction rows. The Section 4 r=0.50 table above is this corrected, verified output.

2026-09-17 – Bootstrap confidence intervals added (10.16f)

A new cell, 10.16f, adds a dataset-level block bootstrap (2,000 resamples, 95% CI) around CHR and mean CG for every method/direction, using only `correction_gains.csv`, already on disk, no new model evaluations. Point estimates were verified to match `chr_table.csv` exactly (max absolute difference 8.33×10^{-17}, floating-point noise). Runtime: ~1.2 seconds. The Section 4 bootstrap table above is this cell's verified output, including the one genuinely new finding it surfaced: `tabpfn_distpfn_t`'s mean CG under Direction A is not statistically distinguishable from zero once dataset-level uncertainty is accounted for.

Explicitly deferred: the matched-test-prior-shift regime

A second test regime (10.21), alongside the existing one, was designed in a prior session: resample the held-out test partition to the same imbalance ratio r as training, instead of leaving it at the dataset's natural prevalence, to isolate how much of a prior-adjusting method's apparent robustness comes from fixing the train/test mismatch this study's design creates versus genuine tolerance to imbalance and label noise. This matters specifically because `tabpfn_threshold` *is* a prior-shift correction by construction, and the DistPFN pair are described in this project's own Related Work as correcting the same kind of shift, so the study's central finding (that the simplest correction is the most robust) has a plausible alternative explanation this check would help rule in or out. Only a cheap one-dataset, one-seed smoke test (10.21a) has been run to verify the mechanism works; the full run (10.21b, scoped to the original 10 datasets × 10 seeds × 4 ratios × 9 method-direction combinations) is estimated at roughly 2.5–4 days of continuous compute, the same order of magnitude as the entire original study's TabPFN-bound compute a second time, and has been deliberately left un-run pending an explicit go-ahead. Skipping it does not corrupt anything else in this document; it leaves the imbalance-versus-prior-shift question an honestly disclosed, empirically untested limitation rather than a resolved one (Section 5).

Net effect on this document's numbers. Every table in Sections 1, 3, and 4 above reflects the current, complete (minus the one documented failure), 26-dataset × 10-seed run: 112,319 of 112,320 expected evaluations. The qualitative headline claim, that the simplest correction (a plain prior-adjusted threshold) is the most robust, held up and if anything strengthened as the dataset pool tripled; the DistPFN pair's failure pattern sharpened into a clean two-direction result rather than the more mixed picture the original 10-dataset run showed. What remains open: reconciling the paper's own prose (dataset count, seed count, r=0.50 narrative) to match this data, deciding whether to run the matched-test regime, and deciding how to handle the one dataset (APS Failure) whose natural imbalance is now more severe than any ratio this study tests directly.

8. References

Every work we cite, alphabetical by first author. Datasets are listed separately at the end; the 16 datasets added in Sections 17–18 of the project's working history are appended to that list with the same UCI Machine Learning Repository citation format as the original 10, since no author/year citation for them exists yet in this project's own bibliography (a gap this update surfaces but does not fix, since inventing one would be exactly the kind of unverified claim this document is meant to avoid).

Methods, models, and prior work

Brier, G. W. (1950). Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1), 1–3.

Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357.

Chen, B., Xia, S., Chen, Z., Wang, B., & Wang, G. (2021). RSMOTE: A self-adaptive robust SMOTE for imbalanced problems with label noise. *Information Sciences*, 553, 397–428.

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *KDD*, 785–794.

Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *JMLR*, 7, 1–30.

Elkan, C. (2001). The foundations of cost-sensitive learning. *IJCAI*, 973–978.

Elor, Y., & Averbuch-Elor, H. (2022). To SMOTE, or not to SMOTE?

Frénay, B., & Verleysen, M. (2014). Classification in the presence of label noise: A survey. *IEEE TNNLS*, 25(5), 845–869.

Friedman, M. (1937). The use of ranks to avoid the assumption of normality implicit in the analysis of variance. *JASA*, 32(200), 675–701.

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. *ICML*, 1321–1330.

Hollmann, N., Müller, S., Eggensperger, K., & Hutter, F. (2023). TabPFN: A transformer that solves small tabular classification problems in a second. *ICLR*.

Hollmann, N., Müller, S., Purucker, L., Krishnakumar, A., et al. (2025). Accurate predictions on small data with a tabular foundation model. *Nature*, 637, 319–326.

Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2), 65–70.

Hu, J., & Ghelichi, M. (2026). Noise immunity in in-context tabular learning: An empirical robustness analysis of TabPFN's attention mechanisms. arXiv:2604.04868.

Khanal, B., Dai, T., Bhattarai, B., & Linte, C. (2024). Active label refinement for robust training of imbalanced medical image classification tasks in the presence of high label noise. *MICCAI*, 37–47.

Koziarski, M., Woźniak, M., & Krawczyk, B. (2020). Combined cleaning and resampling algorithm for multi-class imbalanced data with label noise. *Knowledge-Based Systems*, 204, 106223.

Lee, S. (2026). Mitigating label shift in tabular in-context learning via test-time posterior adjustment. *ICML*. arXiv:2605.04363.

Lemaître, G., Nogueira, F., & Aridas, C. K. (2017). Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets. *JMLR*, 18(17), 1–5.

Li, M., Zhou, H., Liu, Q., Gong, X., & Wang, G. (2024). WRND: A weighted oversampling framework with relative neighborhood density for imbalanced noisy classification. *Expert Systems with Applications*, 241, 122593.

Liu, H., Sheng, M., Sun, Z., Yao, Y., Hua, X.-S., & Shen, H.-T. (2024). Learning with imbalanced noisy data by preventing bias in sample selection. *IEEE Transactions on Multimedia*.

Natarajan, N., Dhillon, I. S., Ravikumar, P., & Tewari, A. (2013). Learning with noisy labels. *NeurIPS*.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., et al. (2011). Scikit-learn: Machine learning in Python. *JMLR*, 12, 2825–2830.

Saerens, M., Latinne, P., & Decaestecker, C. (2002). Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure. *Neural Computation*, 14(1), 21–41.

Scott, C., Blanchard, G., & Handy, G. (2013). Classification with asymmetric label noise: Consistency and maximal denoising. *COLT*, 30, 489–511.

Song, H., Kim, M., Park, D., Shin, Y., & Lee, J.-G. (2022). Learning from noisy labels with deep neural networks: A survey. *IEEE TNNLS*, 34(11), 8135–8153.

Van Hulse, J., & Khoshgoftaar, T. M. (2009). Knowledge discovery from imbalanced and noisy data. *Data & Knowledge Engineering*, 68(12), 1513–1542.

Wilcoxon, F. (1945). Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6), 80–83.

Datasets (UCI Machine Learning Repository) — original 10

Kelly, M., Longjohn, R., & Nottingham, K. The UCI Machine Learning Repository. https://archive.ics.uci.edu/

Lohweg, V. (2012). Banknote Authentication [Dataset]. https://archive.ics.uci.edu/dataset/267

Yeh, I-C. (2008). Blood Transfusion Service Center [Dataset]. https://archive.ics.uci.edu/dataset/176

Grömping, U. (2019). South German Credit Data [Dataset]. https://archive.ics.uci.edu/dataset/522

Hopkins, M., Reeber, E., Forman, G., & Suermondt, J. (1999). Spambase [Dataset]. https://archive.ics.uci.edu/dataset/94

Wolberg, W., Mangasarian, O., Street, N., & Street, W. (1993). Breast Cancer Wisconsin (Diagnostic) [Dataset]. https://archive.ics.uci.edu/dataset/17

Sigillito, V., Wing, S., Hutton, L., & Baker, K. (1989). Ionosphere [Dataset]. https://archive.ics.uci.edu/dataset/52

Haberman, S. (1999). Haberman's Survival [Dataset]. https://archive.ics.uci.edu/dataset/43

Statlog (Heart) [Dataset]. https://archive.ics.uci.edu/dataset/145

Lyon, R. (2015). HTRU2 [Dataset]. https://archive.ics.uci.edu/dataset/372

Mansouri, K., Ringsted, T., Ballabio, D., Todeschini, R., & Consonni, V. (2013). QSAR Biodegradation [Dataset]. https://archive.ics.uci.edu/dataset/254

Datasets (UCI Machine Learning Repository) — 16 more, added Sessions 17–18

Adult [Dataset]. https://archive.ics.uci.edu/dataset/2

Bank Marketing [Dataset]. https://archive.ics.uci.edu/dataset/222

MAGIC Gamma Telescope [Dataset]. https://archive.ics.uci.edu/dataset/159

Online Shoppers Purchasing Intention Dataset [Dataset]. https://archive.ics.uci.edu/dataset/468

Congressional Voting Records [Dataset]. https://archive.ics.uci.edu/dataset/105

Chronic Kidney Disease [Dataset]. https://archive.ics.uci.edu/dataset/336

Default of Credit Card Clients [Dataset]. https://archive.ics.uci.edu/dataset/350

Taiwanese Bankruptcy Prediction [Dataset]. https://archive.ics.uci.edu/dataset/572

Polish Companies Bankruptcy [Dataset]. https://archive.ics.uci.edu/dataset/365

Mushroom [Dataset]. https://archive.ics.uci.edu/dataset/73
Cervical Cancer (Risk Factors) [Dataset]. https://archive.ics.uci.edu/dataset/383
SECOM [Dataset]. https://archive.ics.uci.edu/dataset/179
Arrhythmia [Dataset]. https://archive.ics.uci.edu/dataset/5
Internet Advertisements [Dataset]. https://archive.ics.uci.edu/dataset/51
Seismic-Bumps [Dataset]. https://archive.ics.uci.edu/dataset/266
APS Failure at Scania Trucks [Dataset]. https://archive.ics.uci.edu/dataset/421